

Fiche de Procédure et Déploiement —

SAE 2.03 Gestion Trafic Aérien

Table des matières

1. Prérequis.....	2
2. Infrastructure et Architecture Réseau	2
2.1 Configuration réseau retenue.....	2
3. Déploiement de la VM 1 : Base de Données.....	3
3.1 Installation de MariaDB.....	3
4. Déploiement de la VM 2 : Serveur Web Django	3
4.1 Préparation du système	3
4.2 Transfert du projet depuis le PC hôte	4
4.3 Création et activation de l'environnement virtuel	4
4.4 Préparation des données et des fichiers statiques.....	5
4.5 Configuration du reverse proxy Nginx.....	5
4.6 Lancement de Gunicorn	5
5. Maintenance et Commandes Utiles	6
6. Fonctionnalités et Logique Métier	6
6.1 Règle de disponibilité d'une piste	7
7. Modèle de Données du Projet.....	7
7.1 Données initiales chargées	8
8. Import de Vols par Fichier CSV.....	8
9. Génération des Fiches de Vols.....	9
10. Vérification du Bon Fonctionnement	9
11. Dépannage des Problèmes Fréquents.....	9
12. Conclusion	10

Application web Django 2-Tier pour gérer le trafic aérien : aéroports, pistes, compagnies, avions et vols.

Cette fiche de procédure présente l'infrastructure matérielle et logicielle mise en place, les prérequis, les étapes de déploiement, le fonctionnement métier de l'application et les vérifications à effectuer. Elle a pour objectif de permettre à un évaluateur ou à un nouvel utilisateur de comprendre l'architecture du projet, de le redéployer et de contrôler que les fonctionnalités demandées dans le sujet sont bien présentes.

1. Prérequis

- VirtualBox installé sur le PC hôte.
- Deux machines virtuelles Debian disponibles : une VM pour la base de données et une VM pour le serveur web.
- Accès réseau configuré avec une interface NAT et une interface Host-Only.
- Accès SSH/SCP fonctionnel vers la VM serveur web.
- Projet Django disponible sur le PC hôte.
- Fichier de sauvegarde sauvegarde_donnees.json présent dans le projet.
- Python 3, pip, venv, Nginx, Gunicorn et MariaDB utilisés pour le déploiement.

2. Infrastructure et Architecture Réseau

Schéma logique : PC hôte → redirection NAT 8080 vers 80 → VM serveur web → Nginx → Gunicorn → Django → VM base de données MariaDB sur 192.168.56.10:3306.

Le projet repose sur une architecture 2-Tier (Serveur Web séparé du Serveur de Base de données) virtualisée via VirtualBox.

Composant	Rôle
PC hôte	Permet d'accéder au site depuis un navigateur avec l'adresse 127.0.0.1:8080.
VM serveur web	Héberge Django, Gunicorn et Nginx. Elle reçoit les requêtes HTTP et communique avec la base.
VM base de données	Héberge MariaDB et stocke les données du projet.
Réseau NAT	Permet l'accès depuis le PC hôte via une redirection du port 8080 vers le port 80 de la VM web.
Réseau Host-Only	Permet à la VM web de communiquer avec la VM base de données à l'adresse 192.168.56.10.

2.1 Configuration réseau retenue

- **VM serveur web** : interface NAT avec redirection du port 8080 du PC hôte vers le port 80 de la VM.

- **VM serveur web** : interface Host-Only pour joindre la base de données.
- **VM base de données** : adresse statique 192.168.56.10 sur le réseau Host-Only.
- **MariaDB** : écoute sur le port 3306 afin d'accepter les connexions depuis la VM web.
- **Nginx** : écoute sur le port 80 et transmet les requêtes à Gunicorn sur 127.0.0.1:8000.

3. Déploiement de la VM 1 : Base de Données

3.1 Installation de MariaDB

```
sudo apt update
```

```
sudo apt install -y mariadb-server
```

```
sudo systemctl enable mariadb
```

```
sudo systemctl start mariadb
```

Créer la base et l'utilisateur applicatif dans l'invite SQL MariaDB :

```
CREATE DATABASE sae_aeroport CHARACTER SET utf8mb4;
```

```
CREATE USER 'django_app'@'%' IDENTIFIED BY '<mot_de_passe_django>';
```

```
GRANT ALL PRIVILEGES ON sae_aeroport.* TO 'django_app'@'%';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

Remarque sécurité : le mot de passe réel ne doit pas être diffusé dans la fiche finale. La valeur <mot_de_passe_django> représente un exemple à remplacer uniquement dans l'environnement de déploiement.

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Modifier la ligne bind-address pour écouter sur toutes les interfaces :

```
bind-address = 0.0.0.0
```

Redémarrer le service :

```
sudo systemctl restart mariadb
```

4. Déploiement de la VM 2 : Serveur Web Django

4.1 Préparation du système

```
sudo apt update
```

```
sudo apt install -y python3 python3-pip python3-venv nginx
```

4.2 Transfert du projet depuis le PC hôte

Au lieu d'utiliser Git, le projet est déployé directement par transfert sécurisé (SCP) depuis le PC développeur vers le serveur de production :

```
scp -P 2222 -r C:\Users\<utilisateur>\PycharmProjects\SAE2_03_AEROPORT
toto@127.0.0.1:/home/toto/
```

4.3 Création et activation de l'environnement virtuel

L'environnement virtuel permet d'isoler les dépendances Python du projet afin d'éviter les conflits avec les paquets installés sur le système.

```
cd /home/toto/SAE2_03_AEROPORT
python3 -m venv .venv
source .venv/bin/activate
pip install django gunicorn mysqlclient
nano SAE2_03_AEROPORT/settings.py
```

Vérifier et ajuster ces variables pour la sécurité et la communication :

```
DEBUG = False
ALLOWED_HOSTS = ['127.0.0.1', 'localhost']
CSRF_TRUSTED_ORIGINS = ['http://127.0.0.1:8080']
STATIC_ROOT = BASE_DIR / 'staticfiles'
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'sae_aeroport',
        'USER': 'django_app',
        'PASSWORD': '<mot_de_passe_django>',
        'HOST': '192.168.56.10',
        'PORT': '3306',
    }
}
```

Remarque : la configuration ci-dessus est adaptée à un environnement de test local. Pour une mise en production réelle, il faudrait utiliser un nom de domaine précis, stocker les secrets hors du fichier `settings.py` et limiter les accès réseau.

4.4 Préparation des données et des fichiers statiques

Rassemblement des fichiers statiques pour Nginx :

```
python manage.py collectstatic --noinput
```

Migration des tables dans MariaDB :

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Importation des données de test :

```
python manage.py loaddata sauvegarde_donnees.json
```

4.5 Configuration du reverse proxy Nginx

```
sudo nano /etc/nginx/sites-available/aeroport
```

```
server {
    listen 80;
    server_name _;
    location /static/ {
        alias /home/toto/SAE2_03_AEROPORT/staticfiles/;
    }
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/aeroport /etc/nginx/sites-enabled/
```

```
sudo rm -f /etc/nginx/sites-enabled/default
```

```
sudo chmod a+rx /home/toto
```

```
sudo chmod -R 755 /home/toto/SAE2_03_AEROPORT/staticfiles
```

```
sudo systemctl restart nginx
```

4.6 Lancement de Gunicorn

S'assurer que l'environnement virtuel est activé, puis lancer Gunicorn en arrière-plan :

```
gunicorn SAE2_03_AEROPORT.wsgi:application --bind 127.0.0.1:8000 --daemon
```

5. Maintenance et Commandes Utiles

En cas de mise à jour du code HTML ou Python, il faut relancer Gunicorn.

Action	Commande	Utilisation
Activer l'environnement	<code>source /home/toto/SAE2_03_AEROPORT/.venv/bin/activate</code>	Avant les commandes Python ou Gunicorn.
Arrêter Gunicorn	<code>sudo pkill -9 gunicorn</code>	Après une modification du code ou de Django.
Relancer Gunicorn	<code>gunicorn SAE2_03_AEROPORT.wsgi:application --bind 127.0.0.1:8000 --daemon</code>	Remettre l'application en ligne.
Voir les erreurs Nginx	<code>sudo tail -n 20 /var/log/nginx/error.log</code>	Si la page ne charge pas.
Mettre à jour le CSS	<code>python manage.py collectstatic --noinput</code>	Après modification des fichiers statiques.

6. Fonctionnalités et Logique Métier

- **CRUD complet** : l'application permet de créer, consulter, modifier et supprimer les aéroports, pistes d'atterrissage, compagnies, types d'avions, avions et vols.
- **Gestion des aéroports** : chaque aéroport possède un identifiant, un nom et un pays. Il peut être utilisé comme aéroport de départ ou d'arrivée pour les vols.
- **Gestion des pistes** : chaque piste est rattachée à un aéroport et possède une longueur. Cette longueur est utilisée pour vérifier si un avion peut atterrir sur cette piste.
- **Gestion des compagnies** : chaque compagnie possède un nom, une description et un pays de rattachement. Elle est associée aux avions qu'elle possède.
- **Gestion des types d'avions** : chaque type d'avion contient la marque, le modèle, une description, une image et la longueur minimale de piste nécessaire.
- **Gestion des avions** : chaque avion est rattaché à une compagnie et à un type d'avion. Ces informations sont ensuite utilisées lors de la création des vols.
- **Gestion des vols** : un vol contient un avion, un pilote, un aéroport de départ, une date et heure de départ, un aéroport d'arrivée ainsi qu'une date et heure d'arrivée.
- **Algorithme d'attribution de piste** : lors de l'ajout ou de l'import d'un vol, le système vérifie automatiquement qu'une piste compatible existe à l'aéroport d'arrivée. La piste doit être assez longue pour le type d'avion utilisé.

- **Gestion de l'occupation des pistes** : une piste est considérée comme occupée pendant 10 minutes à l'arrivée d'un avion. Si aucune piste compatible n'est disponible à l'horaire demandé, l'application propose un horaire plus tardif.
- **Import de vols** : l'application permet d'ajouter plusieurs vols à partir d'un fichier CSV préparé à l'avance. Ce mécanisme facilite l'insertion de vols depuis un aéroport.
- **Génération de fiches de vols** : l'utilisateur peut générer une fiche listant les vols prévus au départ ou à destination d'un aéroport pour une date ou une période donnée.

6.1 Règle de disponibilité d'une piste

1. Récupérer l'aéroport d'arrivée du vol.
2. Lister les pistes appartenant à cet aéroport.
3. Conserver uniquement les pistes dont la longueur est supérieure ou égale à la longueur minimale nécessaire pour le type d'avion utilisé.
4. Vérifier les vols déjà prévus sur chaque piste compatible.
5. Considérer qu'une piste est occupée pendant 10 minutes à partir de l'heure d'arrivée prévue.
6. Si une piste est disponible, le vol peut être enregistré à l'horaire demandé.
7. Si aucune piste n'est disponible, l'application recherche le premier créneau ultérieur possible et propose un horaire plus tardif.

7. Modèle de Données du Projet

Entité	Informations principales	Rôle dans l'application
Aéroport	Identifiant, nom, pays	Point de départ ou d'arrivée des vols.
Piste	Numéro, aéroport, longueur	Permet de vérifier la compatibilité avec les avions.
Compagnie	Identifiant, nom, description, pays	Regroupe les avions appartenant à une compagnie.
Type d'avion	Marque, modèle, description, image, longueur minimale de piste	Détermine les contraintes techniques d'atterrissage.

Avion	Identifiant, nom, compagnie, modèle	Appareil utilisé pour planifier un vol.
Vol	Avion, pilote, départ, arrivée, dates et heures	Planifie le trafic aérien entre deux aéroports.

7.1 Données initiales chargées

Type de donnée	Exemples préparés dans la base
Aéroports	Exemples : Paris Charles-de-Gaulle, Londres Heathrow, Madrid-Barajas, Francfort, Amsterdam-Schiphol.
Pistes	Exemples : piste 1 de 3200 m à Paris-CDG, piste 2 de 2800 m à Heathrow, piste 3 de 3500 m à Francfort.
Compagnies	Exemples : Air France, British Airways, Lufthansa, Iberia, KLM.
Types d'avions	Exemples : Airbus A320, Boeing 737, Airbus A350, Boeing 777, avec une longueur minimale de piste associée.
Avions	Exemples : AF-A320-01 pour Air France, BA-B737-01 pour British Airways, LH-A350-01 pour Lufthansa.

8. Import de Vols par Fichier CSV

L'application permet d'insérer plusieurs vols à partir d'un fichier CSV encodé en UTF-8. Le séparateur utilisé est le point-virgule. La première ligne peut contenir les en-têtes afin de faciliter la lecture du fichier. Les dates doivent respecter le format AAAA-MM-JJ HH:MM.

Champ attendu	Description
id_avion	Identifiant de l'avion utilisé pour le vol.
pilote	Nom du pilote affecté au vol.
id_aero_dep	Identifiant de l'aéroport de départ.
date_dep	Date et heure de départ au format AAAA-MM-JJ HH:MM.
id_aero_arr	Identifiant de l'aéroport d'arrivée.
date_arr	Date et heure d'arrivée au format AAAA-MM-JJ HH:MM.

Exemple de fichier CSV :

```
id_avion;pilote;id_aero_dep;date_dep;id_aero_arr;date_arr
3;Martin Dupont;1;2026-06-14 08:30;2;2026-06-14 10:00
```

Avant l'ajout en base, chaque ligne est contrôlée : existence de l'avion, existence des aéroports, cohérence des dates, disponibilité d'une piste compatible à l'arrivée et respect du délai de 10 minutes d'occupation d'une piste.

En cas d'erreur sur une ligne, l'application doit afficher un message explicite indiquant le problème rencontré : avion inexistant, aéroport inexistant, date invalide, piste trop courte ou piste indisponible. Les contrôles évitent l'insertion de vols incohérents dans la base.

9. Génération des Fiches de Vols

Depuis l'interface web, l'utilisateur peut accéder à une page de génération de fiche de vols. Il sélectionne un aéroport, choisit le type de fiche souhaité, puis indique une date précise ou une période. L'application affiche ensuite les vols correspondants sous forme de tableau récapitulatif, utilisable comme fiche de départ ou d'arrivée.

- nom de l'aéroport concerné ;
- période ou date sélectionnée ;
- identifiant du vol ;
- avion utilisé et compagnie associée ;
- nom du pilote ;
- aéroport de départ et aéroport d'arrivée ;
- date et heure de départ ;
- date et heure d'arrivée.

Si aucun vol ne correspond aux critères choisis, l'application affiche un message indiquant qu'aucun résultat n'a été trouvé pour la date ou la période sélectionnée.

10. Vérification du Bon Fonctionnement

- Accéder au site depuis le navigateur du PC hôte avec l'adresse **127.0.0.1:8080**.
- Vérifier que Nginx transmet correctement les requêtes à Unicorn.
- Vérifier que l'application Django communique avec MariaDB sur la VM de base de données.
- Créer, modifier et supprimer au moins un aéroport, une piste, une compagnie, un type d'avion, un avion et un vol.
- Tester l'ajout d'un vol avec une piste compatible disponible.
- Tester l'ajout d'un vol lorsque la piste est déjà occupée afin de vérifier la proposition d'un horaire plus tardif.
- Importer un fichier CSV contenant plusieurs vols.
- Générer une fiche de vols au départ d'un aéroport pour une date donnée.
- Générer une fiche de vols à destination d'un aéroport pour une période donnée.

11. Dépannage des Problèmes Fréquents

Problème rencontré	Vérification ou correction
Erreur 502 Bad Gateway	Vérifier que Gunicorn est lancé sur le port 8000 et relancer le serveur WSGI si nécessaire.
Erreur de connexion à la base	Vérifier l'adresse 192.168.56.10, le port 3306, les droits MariaDB et les identifiants Django.
CSS non chargé	Relancer collectstatic, vérifier le dossier staticfiles et contrôler la configuration Nginx.
Migration impossible	Vérifier que l'environnement virtuel est activé et que les dépendances Python sont installées.
Import CSV refusé	Contrôler le séparateur point-virgule, le format des dates et l'existence des identifiants utilisés.

12. Conclusion

Cette fiche répond aux attentes du rendu en présentant l'infrastructure matérielle et logicielle, la configuration réseau, les étapes de déploiement, le fonctionnement métier de l'application et les contrôles nécessaires après installation. Elle montre également que le projet respecte les exigences du sujet : gestion complète des entités, vérification de la disponibilité des pistes, import de vols par fichier et génération de fiches de vols selon un aéroport et une période.